

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

Informatik, Universität Trier

Zeitplan bis zum Jahresende

08.01.2026	Datenflussanalyse	Übungsblatt „Datenflussanalyse“
13.01.2026	Datenflussanalyse von TRIPLA	
15.01.2026	Projekt IV (Kontrollflussanalyse)	Abgabe Übungsblatt „Datenflussanalyse“
20.01.2026	Code Clone Detection	
22.01.2026	Reverse Engineering, Erkennung von Entwurfsmustern	Abgabe Projekt IV
27.01.2026	Gastvortrag: „Programm Analysis and Transformation from a Practitioner's Point of View“ (Jonas Eckstein, itestra)	Projekt V (Datenflussanalyse)
29.01.2026	Programmtransformationen	Übungsblatt „Programmtransformationen“
03.02.2026	Partielle Evaluation von TRIPLA	
05.02.2026	Higher-Order TRIPLA	
10.02.2026	Details zum Portfolio + Tipps zu Projekt V	Abgabe + Übung ÜTRAFO
17.03.2026	Abgabe Portfolio inkl. Projekt V	



Erkennen von Kodeklonen (code-clone detection)

```

1 public class Gestern {
2     static boolean kleiner(int a, int b) { return a<b; }
3     static boolean groesser(int a, int b) { return a>b; }
4     static boolean gleich(int a, int b) { return a==b; }
5
6     public static void main(String argv[]) {
7         for(int i=0; i<100; i++)
8             for(int j=0; j<100; j++) {
9                 if (groesser(i,j))
10                    System.out.println("i="+i+" j="+j);
11             }
12     }
13
14
15 }
16

```

```

1 import java.io.FileOutputStream;
2 import java.io.PrintWriter;
3 import java.io.IOException;
4
5 public class Heute {
6     static boolean kleiner(int a, int b) { return a<b; }
7     static boolean groesser(int x, int y) { return x>y; }
8     static boolean gleich(int a, int b) { return (a-b)==0; }
9
10    static boolean schreibeLog=true;
11
12    static void log(int i) {
13        try (FileOutputStream fout
14            = new FileOutputStream("zahlen.log",true);
15            PrintWriter bout = new PrintWriter(fout);) {
16            bout.println("i=" + i);
17        } catch (IOException e) {
18            System.err.println(e.getMessage());
19        }
20    }
21
22    public static void main(String argv[]) {
23        for(int i=0; i<100; i++)
24            for(int j=0; j<100; j++) {
25                if (schreibeLog) log(i);
26                if (schreibeLog) log(j);
27                if (groesser(i,j))
28                    System.out.println("i="+i+" j="+j);
29            }
30        for(int i=99; i>=0; i--)
31            for(int j=99; j>=0; j--) {
32                if (schreibeLog) log(i);
33                if (schreibeLog) log(j);
34                if (groesser(i,j))
35                    System.out.println("i="+i+" j="+j);
36            }
37    }
38 }
39

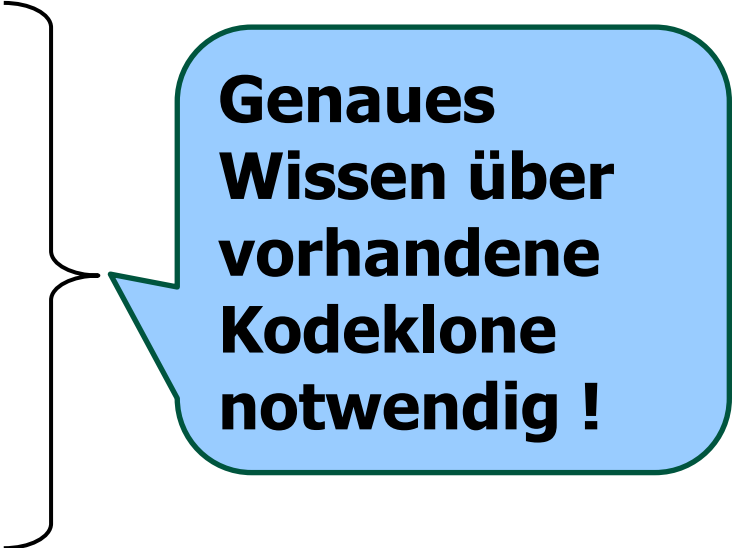
```

Wie entstehen Kodeklone?

- **Cut&Paste:** Eine neue Funktionalität soll eingebaut werden. Der Programmierer sucht nach einem passenden Fragment im existierenden Code, kopiert und modifiziert dieses.
- **Inlining:** Um die Laufzeit zu verbessern (bei nicht-optimierenden Compilern) wird ein Methodenaufruf durch eine (optimierte) Kopie des Methodenrumpfs ersetzt.
- **Unabhängig:** Ähnliche Probleme werden (von unterschiedlichen Programmierern) durch ähnliche Programmstrukturen gelöst.

Probleme durch Kodeklone

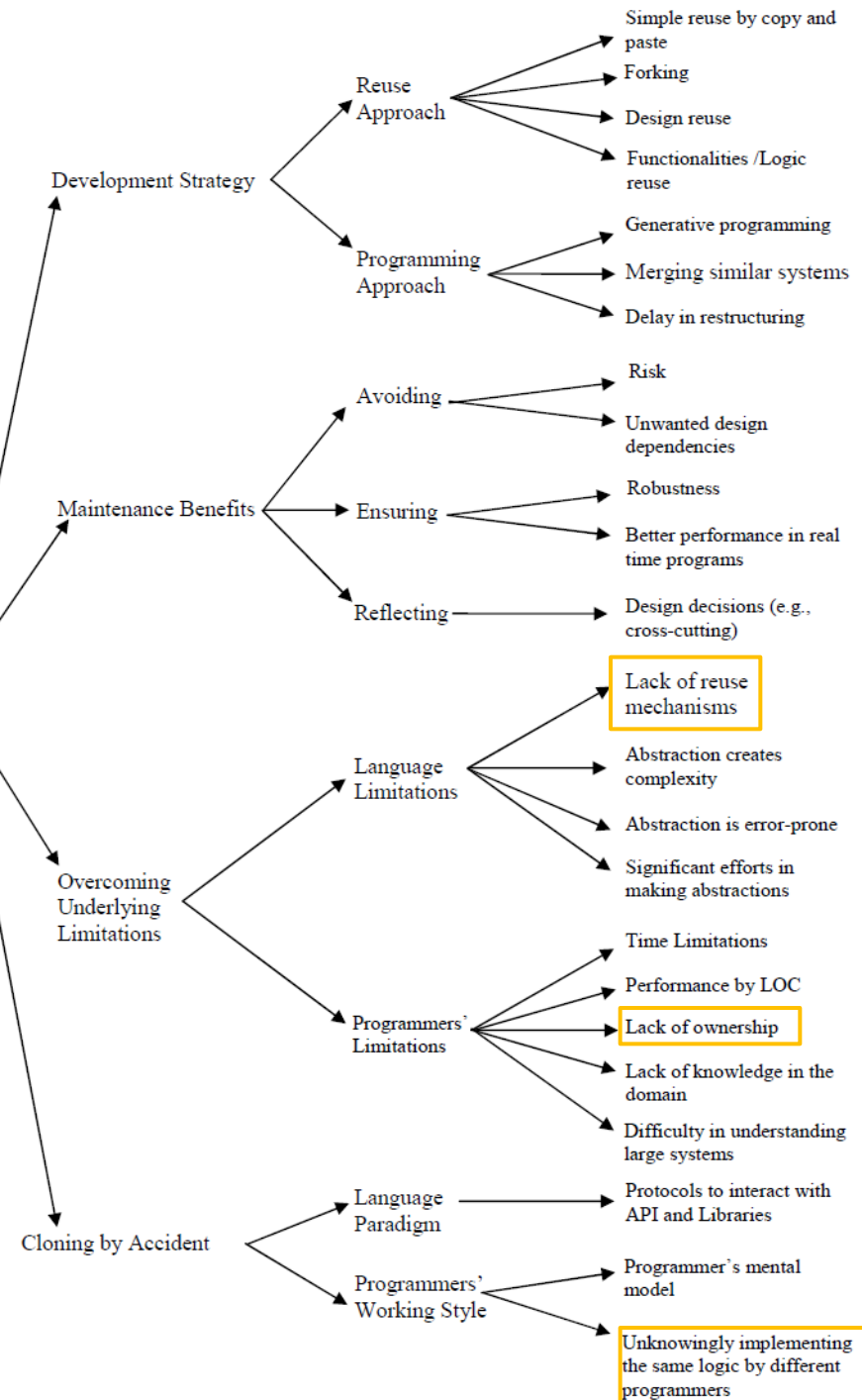
- Duplizierter Code kann die gleichen Fehler enthalten.
- Duplizierte Code muss möglicherweise zusammen geändert werden.
- Duplizierter Code muss „mehrfach“ verstanden werden.
- Mehr Programmcode
- Studien: 5-25% redundanter Code



**Genaueres
Wissen über
vorhandene
Kodeklone
notwendig !**

- **Quelle: A Survey on Software Clone Detection Research,**
Chanchal Kumar Roy and James R. Cordy, September 26, 2007, Technical Report No. 2007-541, School of Computing, Queen's University at Kingston, Ontario, Canada

24 Reasons for Cloning



Kodeklone: Definition

”

Clones are segments of code that are similar according to some definition of similarity.

—Ira D. Baxter

“

Die **ähnliche Kopie** eines Programmfragments wird Kodeklon genannt.

Beispiele für Klone

```
i = length (l);  
if (i < 3) {...}
```

```
i = length (l);  
if (i < 3) {...}
```

```
i = length (l);  
if (i < 3) {...}
```

```
k = length (j);  
if (k < x) {...}
```

```
i = length (l);  
if (i < 3) {...}
```

```
k = length (j);  
if (j < 3) {...}
```

```
i = length (l);  
if (i < 3) {...}
```

```
k = length (j);  
f();  
if (k < x) {...}
```

```
x = x + 1;
```

```
x++;
```

Klassifikation von Klonen

- **Typ 1: Exakte Kopie**
 - Keinerlei Veränderung an der Kopie (bis auf White Space und Kommentare).
 - Z.B. Inlining von Hand.
- **Typ 2: Kopie mit Umbenennungen** (parametrisierte Übereinstimmung)
 - Bezeichner werden in der Kopie umbenannt.
 - Z.B. „Wiederverwendung“ einer (generischen) Funktion von Hand
- **Typ 3: Kopie mit weiteren Modifikationen**
 - Code der Kopie wird abgeändert, nicht nur Bezeichner umbenannt.
 - Z.B. „Erweiterung“ einer Funktion.
- **Typ 4: Semantische Klone**
 - Verschiedene Implementierungen desselben Konzepts.

Beispiele für Klontypen

<pre>i = length (l); if (i < 3) {...}</pre>	<pre>i = length (l); if (i < 3) {...}</pre>	Typ 1
<pre>i = length (l); if (i < 3) {...}</pre>	<pre>k = length (j); if (k < x) {...}</pre>	Typ 2
<pre>i = length (l); if (i < 3) {...}</pre>	<pre>k = length (j); if (j < 3) {...}</pre>	zwei Typ 2 bzw. ein Typ 2 mit inkonsistenter Umbenennung
<pre>i = length (l); if (i < 3) {...}</pre>	<pre>k = length (j); f(); if (k < x) {...}</pre>	Typ 3
<pre>x = x + 1;</pre>	<pre>x++;</pre>	primitiver Typ 4

Grundlage zum automatischen Finden von Kodeklonen

- Text
 - z.B. *Unix-Diff*, zeilenweise
- Token
 - z.B. *Pattern Matching auf Basis der Lexeme [Baker]*
- Syntax
 - z.B. *Teilbäume im abstrakten Syntaxbaum [Baxter et al.]*
- quantifizierbare Merkmale
 - z.B. *Vergleich von Codemetriken [Mayrand et al.]*
- Semantik
 - z.B. *Isomorphic Subgraph Matching auf Program Dependence Graph (CFG+DFG) [Krinke]*

Probleme bei der Erkennung

- Typ 1:
 - Jede Zeile/Funktion/Datei muss **mit jeder anderen** Zeile/Funktion/Datei **verglichen** werden. Wie kann quadratischer Aufwand vermieden werden?
- Type 2:
 - Wie kann **von Bezeichnern geeignet abstrahiert** werden? Soll die Umbenennung konsistent sein?
- Typ 3:
 - Geklonte Codefragmente können zu größeren Klonen zusammengefasst werden. Codefragmente müssen **nicht direkt zusammenhängend** sein.
- Typ 4:
 - Wie entscheidet/rät man, ob zwei Codefragmente das Gleiche berechnen?
Halteproblem?

Token-basierte Kodeklone-Erkennung

Verfahren von Brenda S. Baker

- Verfahren basiert auf Token (Lexemen).
- Vermeidung des quadratischen Aufwands:
 - quasi-parallele Suche in einer Programmrepräsentation, die jedes mögliche Anfangsstück des Programms enthält.
- Abstraktion von Bezeichnern:
 - Bezeichner werden auf relative Positionen abgebildet.
- Typ 3 Klon:
 - Separater Schritt am Ende



Quelle: Brenda S. Baker. A Program for Identifying Duplicated Code. *Computing Science and Statistics*, Vol. 24, pp 49-57, 1992.

Verfahren von Baker

- Für **jede Codezeile** wird eine Zeichenkette aus Parametersymbolen und Nichtparametersymbolen generiert (Parameter-String oder kurz P-String):
 - Struktur der Zeile wird auf eindeutiges Nichtparametersymbol abgebildet (Funktork: $\alpha, \beta, \dots$)
 - **Bezeichner und Literale** werden in Argumentliste erfasst
 - Ergebnis ist: (Funktork; Argumentliste)
 - Beispiel: $x = x + y \rightarrow ([ID] = [ID] + [ID]; x, x, y) \rightarrow \alpha x x y$
- Die Konkatenation der P-Strings aller Codezeilen repräsentiert das Programm.

```
j = length(l);  
if (j < 3) { x = x + y ; }
```

$\alpha \rightarrow [ID] = [ID] ([ID]) ; \text{if}$
 $\beta \rightarrow ([ID] < [LIT]) \{ [ID] = [ID] + [ID] ; \}$
 $\rightarrow \alpha j \text{ length } l \beta j 3 x x y$

Verfahren von Baker

Kodierung $\text{prev}(s)$ jedes P-Strings s :

- Erstes Vorkommen eines Bezeichners erhält Nummer 0.
- Jedes weitere Vorkommen erhält den **relativen Abstand zum vorherigen Vorkommen** (Funktoren mitgezählt).
- Beispiel:

$$\text{prev} (\alpha \text{ j length l } \beta \text{ j 3 x x y}) = \alpha \text{ 0 0 0 } \beta \text{ 4 0 0 1 0}$$

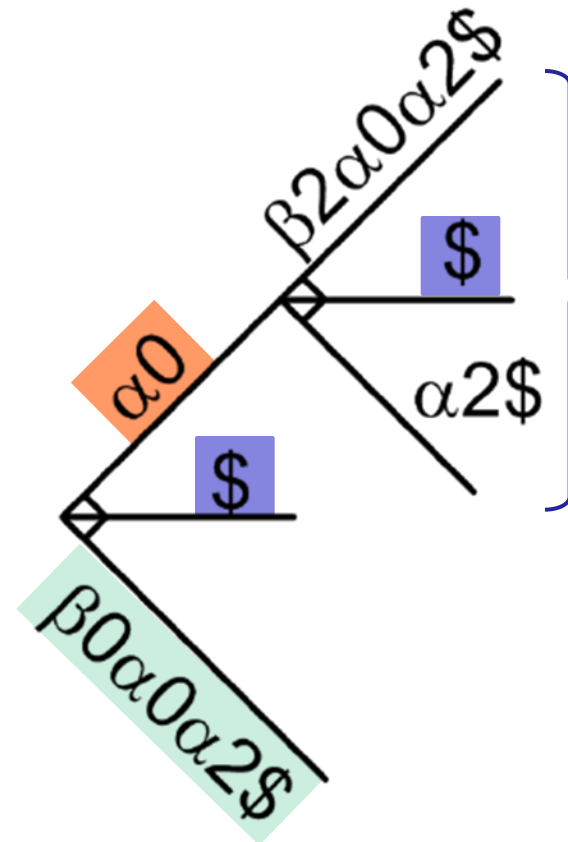
- Abstraktion der Bezeichner, jedoch nicht ihrer Reihenfolge!
- S ist P-Match von T genau dann, wenn $\text{prev}(S) = \text{prev}(T)$
- Beispiel:

$$x = x + y \text{ und } a = a + b \text{ sind ein P-Match, da } \alpha 010 = \alpha 010$$

Präfixbaum: **abbaba\$**

Verfahren von Baker

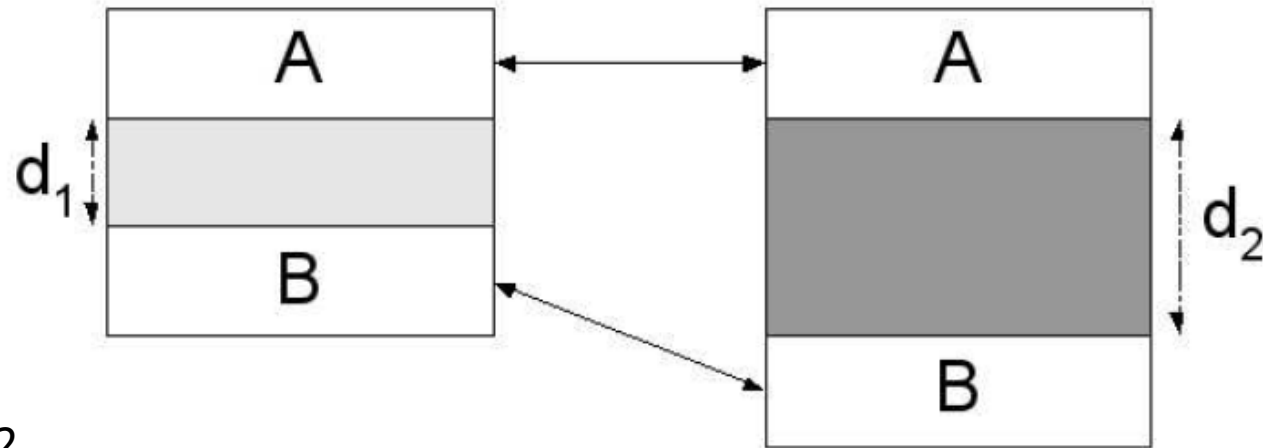
- P-Suffix-Baum $S = s_1 \dots s_n \$$
 - Sei $S_i = s_i s_{i+1} \dots s_n \$$ das i-te Suffix von S ($\$$ ist Endezeichen).
 - Der P-Suffix-Baum von S enthält alle $\text{prev}(S_i)$ zu den Suffixen des P-Strings S.
 - Beispiel: $S = \alpha y \beta y \alpha x \alpha x$
 - $\text{prev}(S_1) = \alpha 0 \beta 2 \alpha 0 \alpha 2 \$$
 - $\text{prev}(S_2) = 0 \beta 2 \alpha 0 \alpha 2 \$$
 - $\text{prev}(S_3) = \beta 0 \alpha 0 \alpha 2 \$$
 - $\text{prev}(S_4) = 0 \alpha 0 \alpha 2 \$$
 - $\text{prev}(S_5) = \alpha 0 \alpha 2 \$$
 - $\text{prev}(S_6) = 0 \alpha 2 \$$
 - $\text{prev}(S_7) = \alpha 0 \$$
 - $\text{prev}(S_8) = 0 \$$
 - $\text{prev}(S_9) = \$$



Der Klon $\alpha 0$
kommt dreimal vor.

Verfahren von Baker

- Zusammenfassen von P-Matches
 - **pro Zeile** wird ein P-String berechnet und in P-Suffix-Baum eingetragen
 - Klone lassen sich im Baum durch Verzweigungen identifizieren.
 - Benutzer legt die **Mindestzeilenlänge** der Klone fest; entspricht der Anzahl von Funktoren.
 - Gefundene P-Matches werden, wenn möglich, zusammengefasst:



- Strategien:
 - nur wenn $d_1 = d_2$
 - wenn $\max(d_1, d_2) \leq d_{\max}$

Verfahren von Baker

- Vorteile
 - Sehr schnell, da token-basiert
 - ignoriert Leerzeichen, Leerzeilen oder Kommentare.
 - Der Ansatz ist weitgehend unabhängig von der jeweiligen Programmiersprache (nur Regeln für Bezeichner/Tokens notwendig).

- Nachteile: erkennt keine

- äquivalenten Ausdrücke mit kommutativen Operatoren

<code>x = x + y</code>	<code>x = y + x</code>
------------------------	------------------------

- gleichen Anweisungsfolgen, die verschieden umgebrochen wurden

<code>if (a > 1) {x = 1;}</code>	<code>if (a > 1) {x = 1;}</code>
-------------------------------------	---

- gleichen Teilausdrücke

<code>if (sp > 0)</code>	<code>if (sp > 0 && s[sp] != a)</code>
-----------------------------	---

Precision and Recall

- **Precision p**
 - Es waren p Prozent der gefundenen Kandidaten auch wirkliche (bekannte) Kodeklone.
- **Recall r**
 - Es wurden r Prozent der wirklichen (bekannten) Kodeklone gefunden.

Table 14: Higher level comparison of the detection approaches

Approach	Portability	Precision	Recall	Scalability
Line-based	High, needs lexer at most	100%, No false positives as checks for exact copies	Low, only finds exact copies	Depends on Comparison algorithms
Parameterized line based	High, needs lexer at most	Medium, may return false positives	Medium, can detect only exact and parameterized clones	Disagreement among researchers
Token-based	Medium, Needs lexer transformation rules	Low, due to normalization and/or transformation returns many false positives	High, can detect most clones	High with suffix-tree algorithm
Parse-tree based	Low, needs parser	High, parse-tree considers structural info also	Low, cannot detect all types of clones, however, several approaches taken to overcome this	Depends, how comparison is made
PDG-based	Low, needs PDG generator	High, considers structural and semantic info too	Medium, cannot detect all clones	Low, graph matching is costly
Metrics-based	Low, needs parser/PDG generator to get metrics values	Medium, two code blocks with similar metrics values may not be same	Medium, cannot detect many clones	High, after getting metrics values, needs only to compare the metrics values for each begin-end block
AST + Suffix tree	Low, needs parser	High, considers structural info	Low, cannot detect all clones	High, applies suffix tree comparison on AST nodes

Studie zu Precision and Recall

- Vergleichene Systeme in der Studie
 - CCFinder: Token-basiert
 - CloneDr: AST-basiert (Baxter)
 - Covet: Metrik-basiert (Mayrand)
 - JPlag: Token-basiert
 - Moss: Token-basiert

	CCFinder	CloneDr	Covet	JPlag	Moss
Precision	72%	100%	63%	82%	73%
Recall	72%	9%	19%	12%	10%

Problem der Studie:

Wer definiert, was ein Kodeklone ist?

Elizabeth Burd, John Bailey. Evaluating Clone Detection Tools for Use during Preventative Maintenance. In *Proceedings of the 2nd IEEE International Workshop on Source Code Analysis and Manipulation (SCAM'02)*, pp. 36 - 43, Montreal, Canada, October 2002.

... und zum guten Schluß

- Kodeklone sind nicht so schlimm wie ihr Ruf:

*"We find that, first, the great majority of bugs are not significantly associated with clones. Second, we find that **clones may be less defect prone than non-cloned code**. Finally, we find little evidence that clones with more copies are actually more error prone. Our findings **do not support the claim that clones are really a 'bad smell'**. Perhaps we can clone, and breathe easy, at the same time."*

Quelle: **Clones: What is that Smell?** Foyzur Rahman, Christian Bird, and Premkumar Devanbu. In Proceedings of the Seventh Working Conference on Mining Software Repositories, (Best Paper Award), Cape Town, South Africa, 2010.